

Fraud Intelligence Platform - Project Guide (Zero to Hero)

A plain-English, end-to-end explanation of what this project is, what it does, how it was built, and how to use the web interface you are looking at right now.

1. What is this project about?

This is a **fraud detection system for online payments**.

Imagine a website that processes thousands of card transactions every day. A small fraction of those transactions are fraudulent (stolen cards, stolen accounts, etc.). The business wants to catch fraud **before** money leaves, but without annoying honest customers by flagging too many of them.

This project builds a machine-learning model that, given the details of a transaction, outputs a **fraud score between 0 and 1**:

- score near 0 -> looks clean
- score near 1 -> looks fraudulent

If the score is above a chosen threshold (here `0.798`), the transaction is sent for **manual review**. Otherwise it is allowed.

That is the whole point of the product: **automatically score transactions for fraud, explain why, and let a human review the risky ones.**

2. What does it actually do?

In one sentence: it takes a transaction's engineered features, scores it with an XGBoost model, and tells you `CLEAN` or `FRAUD RISK` with the reasons.

Concretely the system can:

1. Download and prepare a real, public fraud dataset (IEEE-CIS).
2. Build 416 "features" (numeric descriptions) out of each transaction.
3. Train and compare two models: a simple baseline and a stronger XGBoost model.
4. Pick the best model using a strict, fair "promotion gate".
5. Serve the chosen model through a web API.
6. Show a web page where a human can load a real transaction, tweak it, and see the model's verdict and risk factors.
7. Watch the model in production for data drift and bad behavior.
8. Explain every single prediction with the features that pushed the score up or down.

Everything is reproducible: the same commands always produce the same results.

3. The dataset and the real-world problem

We use the **IEEE-CIS Fraud Detection** dataset (a well-known public competition

dataset). It contains:

- `Transaction` table: amount, time, and hundreds of behavior/context fields.
- `Identity` table: device and browser information (when available).

Key facts you should know:

- Fraud is **rare**: only about 3.5% of transactions are fraudulent. This is called a *class imbalance*, and it makes the problem harder.
- Many columns are masked (`V1`..`V339`). They are real engineered signals but their meaning is hidden, so we treat them as opaque numbers.
- The most important practical rule: **we must predict the future using only the past.** If we accidentally let future information leak into training, the model looks amazing in tests but fails in reality.

That last point shaped the entire design.

4. How we built it (the pipeline, rough level)

The project is built as a sequence of small, independent steps (called "stages"). Each stage reads the previous stage's output and writes its own. This is managed by **DVC** (Data Version Control), so the whole thing is reproducible.

Here is the high-level flow:

```
Raw data (Kaggle CSVs)
  |
  v
[1] Ingest          -> clean join of transactions + identity
  |
  v
[2] Feature build   -> 416 leakage-safe features (historical aggregates)
  |
  v
[3] Train baseline  -> Logistic Regression (the "floor")
  |
  v
[4] Train candidate -> XGBoost (the "ceiling")
  |
  v
[5] Compare + gate  -> pick the winner only if it is safe and fair
  |
  v
[6] Register model  -> save the champion to MLflow
  |
  v
[7] Explain         -> SHAP risk factors for any transaction
  |
  v
[8] Serve           -> FastAPI web service + web UI
  |
  v
[9] Monitor         -> Prometheus metrics + drift checks
```

|
v

[10] Report + UI -> final report, web UI, real-time demo

The most important technical idea: leakage-safe features

The model is not fed the raw transaction. It is fed **features** - numbers derived from the transaction. Some of the most powerful features are "historical aggregates", for example:

- ``card1_hist_fraud_rate``: across all past transactions using this card, what fraction were fraudulent?
- ``card1_hist_amt_mean``: what is the usual transaction amount for this card?
- ``card1_hist_count``: how many times have we seen this card before?

These are computed **only from earlier transactions** (strictly past), so the model never cheats by looking at the future. This is the same idea a real bank's "feature store" uses in production.

The most important evaluation idea: temporal split

We split the data by **time**, not randomly:

- Train on the oldest transactions.
- Validate on the next window.
- Test on the newest window (the "hold-out").

If the model works on the newest, unseen window, we trust it. This is far more honest than a random split.

The promotion gate

We do not promote a model just because it has the highest score. The gate requires:

- It beats the baseline on the hold-out window.
- It meets minimum precision and recall at the operating threshold.
- It performs acceptably on important subgroups (e.g., transactions where identity data is present vs absent).

Only if all of those pass does the model become the "champion".

5. Key functionalities

- **Reproducible data pipeline** (DVC): download, ingest, feature build.
- **Model training + comparison**: baseline vs XGBoost, with MLflow tracking.
- **Promotion gate + model registry**: only safe models go to production.
- **Explainability**: every prediction comes with its top risk factors (SHAP).
- **REST API**: `/health``, `/predict``, `/predict/batch``, `/metrics``, `/sample``, `/demo/features``.
- **Web UI**: load a real transaction, edit it, score it; plus a real-time

feature demo.

- **Monitoring**: live Prometheus metrics and periodic drift detection (Evidently).

6. Technologies used

- **Python 3.11** - the language.
- **pandas / numpy** - data handling.
- **scikit-learn** - the baseline model and metrics.
- **XGBoost** - the winning gradient-boosted tree model.
- **FastAPI / uvicorn** - the web service.
- **Prometheus + Evidently** - monitoring and drift.
- **MLflow** - experiment tracking and model registry.
- **DVC** - reproducible data + pipeline.
- **Docker** - packaging the service into a container image.

You do not need to know all of these to understand the project. The important ones for "what is happening" are: **pandas** (data), **XGBoost** (model), **FastAPI** (service), **Docker** (packaging).

7. How it works in practice (running it locally)

Everything is packaged into a **Docker image** called `fraud-intel-api:latest`. When you run that image, Docker starts a small web server inside a container and maps it to your machine's port 8000.

So on your own computer ("localhost") the flow is:

```

Your browser -> http://localhost:8000 -> web server inside Docker container
                                         |
                                         v
                                         XGBoost model
  
```

Steps to run (the short version):

1. Start **Docker Desktop** (on this machine the Docker engine sometimes stops, so start it first).
2. In a terminal, from the project folder, run:
``docker compose up -d``
3. Open **http://localhost:8000** in your browser.

That is it. The model, the features, and the web page are all inside the image, so there is nothing else to install.

8. The Web UI - a full walkthrough

This is the part you are already using. Here is exactly what every box and button does.

Section A: "Check a transaction"

When the page loads, it shows a card titled with a transaction id like `sample-105`.

"New random transaction" button

- Loads a *real* transaction taken from the training data.
- It contains all 416 engineered features, including the true label (fraud or not) which is hidden from you but used to show "did the model agree?".
- Every click loads a different real transaction, so the numbers in the form change. That is why you see, for example, the transaction id or values jump from one sample to another each time you click it.

The 12 number boxes (the placeholders)

- These are the 12 features that most influence the model for this dataset.
- Each box is editable: you can type any number to simulate "what if this value were different?".
- When you load a random transaction, the boxes are filled with that transaction's real values.

"Clear to blank" button

- Empties all the boxes so you can type a **manual** transaction from scratch.
- Any feature you leave blank defaults to 0 before scoring.

"Check fraud" button

- Sends the current values to the model.
- The page then shows:
 - A big verdict: **FRAUD RISK** (red) or **CLEAN** (green).
 - The model score (e.g. `0.5487`) and the threshold (`0.798`).
 - The **top risk factors**: the features that pushed the score up the most, with their SHAP contribution (e.g. `card1\_hist\_fraud\_rate (+1.3854)`).
 - Whether the model agreed with the real label (since this was a real transaction).

Section B: "Real-time feature demo"

This is the educational part. It shows how raw data becomes model features.

Three boxes: `card1`, `addr1`, `TransactionAmt`

- `card1` and `addr1` are raw identity keys (like a card id and an address id).
- `TransactionAmt` is the raw transaction amount.
- These are the kind of raw fields a real payment system would send.

"Recompute history -> re-predict" button

- The app takes those three raw values and, using the training history, computes the model's historical-aggregate features **live** - exactly like an offline feature store would.
- Example output you will see:
 - `card1=12715: found in training history (3 past txns, fraud rate 0.0000)`
 - Then it re-scores the transaction with those freshly computed features and shows the new verdict and score (and the base score, so you can compare).

Why this matters: in a real bank, a separate "feature store" does this

computation before the model is called. This demo makes that hidden step visible and interactive.

9. Practical use cases

- **Fraud analyst workstation**: load a suspicious transaction, see the model's reasoning, decide whether to block it.
- **Teaching / interview demo**: show, end-to-end, how a fraud model is built, evaluated, served, explained, and monitored.
- **Starting point for production**: the API is the same shape a real scoring service would use; only the surrounding infrastructure (auth, rate limiting, a real feature store) would be added.

10. Project structure (brief)

```
src/fraudintel/      # all the real code (data, features, models, serving)
scripts/            # runnable pipeline steps used by DVC
tests/              # automated tests that prove the code works
configs/            # settings (training, thresholds)
docs/               # design + this guide
data/               # raw / processed data (DVC tracked, not committed)
artifacts/          # models, reports, UI samples (DVC tracked)
docker-compose.yml  # starts the service container
Dockerfile          # builds the image
dvc.yaml            # defines the reproducible pipeline stages
```

You normally do not need to touch any of this to use the UI - it is all inside the Docker image.

11. Quick start (copy-paste)

Start Docker Desktop, then:

```
cd "D:\ML & AI\ML Projects\fraud-intelligence-platform"
docker compose up -d
```

Then open **http://localhost:8000**.

To stop it:

```
docker compose down
```

To rebuild the image from source after changing code (takes ~10-15 minutes):

```
docker build -t fraud-intel-api:latest .
```

12. Honest limitations

- **Identity gap**: the model is much better when identity data is present (card/address known) than when it is absent. This is a real fairness/risk finding we keep visible rather than hide.
 - **Feature store needed in production**: the API accepts already-engineered features. Computing them from raw transactions in real time is the job of an upstream feature store (the UI demo shows what that step does).
 - **Rare events**: because fraud is rare, even a good model still misses some fraud and flags some honest customers. The threshold is a business choice (how much review capacity you have).
-

13. Glossary (beginner terms)

- **Feature**: a single numeric input to the model (e.g. transaction amount).
 - **Model**: the trained math that turns features into a score.
 - **XGBoost**: a popular, powerful type of decision-tree model.
 - **Score**: the model's output, a number from 0 (clean) to 1 (fraud).
 - **Threshold**: the cut-off above which we review the transaction.
 - **SHAP / risk factor**: a way to explain which features pushed the score up.
 - **Drift**: when real-world data starts looking different from training data, so the model may degrade.
 - **DVC**: tool that makes the data pipeline reproducible.
 - **Docker**: packages the app + model into a portable container.
 - **API**: a programmatic way for other software to call the model.
 - **UI**: the web page a human uses (what you are looking at).
-

14. One-paragraph summary you can say out loud

"We built an end-to-end fraud detection system. It ingests a public payments dataset, engineers leakage-safe features, trains and compares models with a strict time-based evaluation, and promotes the best one through a safety gate. The winning XGBoost model is served through a web API and a web UI. The UI lets a person load a real transaction, tweak its values, and see the fraud verdict plus the reasons; a real-time demo shows how raw identity and amount fields become the model's historical features. The whole thing is reproducible with DVC, tracked with MLflow, monitored for drift, and packaged in Docker so it runs with a single command."

Generated as part of the Fraud Intelligence Platform documentation.